

PROPOSED TRANSFERABLE SOFTWARE / IP SCOPE

SUBJECT TO EXECUTED TRANSACTION AGREEMENTS

STRICT PAPER TRADING (\$0.00 CAPITAL RISK)

1. EXECUTIVE TRANSACTION SCOPE OVERVIEW

The proposed transaction is structured as a software and intellectual property asset acquisition, subject to executed definitive transaction agreements. Under the proposed transferable software/IP scope, the acquiring entity would receive proprietary source codebases, quantitative algorithms, automated test harnesses, database schemas, and closing due diligence dossiers. To eliminate third-party account transfer friction and ensure clean corporate separation, cloud hosting instances, payment processor accounts, and external market data keys are independently provisioned by the buyer under their corporate entity.

2. PROPOSED TRANSFERABLE ASSETS VS. BUYER-PROVISIONED OBLIGATIONS

Asset / Subsystem	Proposed Transferable Software / IP Scope	Buyer-Provisioned External Obligations
Application Source Code	Proposed Transferable: Complete Git repository (React SPA, FastAPI backend, CLI tooling).	None. Transferred via Git repository transfer or mirror.
Domain Logic Layer	Proposed Transferable: 21 pure internal domain packages (strategies, backtesting, risk).	None. Self-contained in repository with zero external framework imports.
Automated Test Harness	Proposed Transferable: Documented baseline of 4,260 tests across 103 suites (99.4% coverage).	None. Executes locally or in CI/CD with Python 3.11+.
Database Architecture	Proposed Transferable: SQLAlchemy 2.0 Async models and 15 linear Alembic migrations (29 tables).	Buyer provisions target PostgreSQL 16 database (Docker or cloud PaaS).
Disaster Recovery Scripts	Proposed Transferable: Tested restore runbooks and shell scripts (~7.2s historical benchmark).	Buyer configures automated backup snapshot schedules on cloud database.
Due Diligence Dossiers	Proposed Transferable: All 18 closing dossiers under docs/acquisition/.	None. Stored directly within repository markdown tree.
PaaS Cloud Blueprint	Proposed Transferable: Declarative render.yaml blueprint and Docker container specs.	Buyer-Provisioned: Buyer creates Render account (~\$14/mo baseline hosting config estimate).
Commercial Billing Engine	Proposed Transferable: Stripe webhook listeners, subscription tiers, and checkout routers.	Buyer-Provisioned: Buyer establishes Stripe merchant account & supplies live API keys.
Market Data Ingestion	Proposed Transferable: MockMarketDataProvider and TwelveDataMarketDataProvider adapters.	Buyer-Provisioned: Buyer secures TwelveData API key if live market quotes desired.
Broker Sandbox Layer	Proposed Transferable: PaperExecutionAdapter matching engine and OandaBrokerAdapter.	Buyer-Provisioned: Buyer registers OANDA Practice sandbox account for external testing.
Web Domain & SSL	Proposed Transferable: Reverse proxy routing, CSP security policies, and SPA build configuration.	Buyer-Provisioned: Buyer registers proprietary custom domain and configures DNS routing.

3. INDEPENDENT ACCOUNT PROVISIONING PROTOCOL (5 STEPS)

- Step 1: Code Repository Transfer**

 - Transfer GitHub repository or clean git bundle to buyer organization.
 - Verification of commit signature chain and linear history through baseline commit d5908d0a.
- Step 2: Cloud Infrastructure Initialization**

 - Buyer creates organization on Render PaaS and connects repository.
 - Deploys render.yaml blueprint (~\$14/mo baseline hosting config estimate).
- Step 3: Environment Secret Injection**

 - Render generates ORION_JWT_SECRET_KEY & internal PostgreSQL/Redis URLs.
 - Buyer injects optional external vendor API keys (TwelveData, Stripe).
- Step 4: Database Schema Initialization**

 - Automated preDeployCommand executes python scripts/deploy/migrate.py.
 - Applies 15 linear Alembic migrations across 29 tables; verified ~7.2s DR restore benchmark.
- Step 5: Custom Domain Association & TLS Edge Routing**

 - Buyer registers corporate custom domain and configures DNS CNAME records pointing to Render edge.
 - Automated Let's Encrypt TLS certificate provisioning and security header enforcement (HSTS, CSP, X-Frame DENY).

4. PERMISSIVE OPEN-SOURCE LICENSING AUDIT & DEPENDENCY GOVERNANCE (DOSSIER 12)

- **Backend Stack:** Python 3.11+, FastAPI (MIT), Pydantic v2 (MIT), SQLAlchemy 2.0 (MIT), Alembic (MIT), Uvicorn (BSD-3-Clause).

• **Frontend Stack:** React 18 (MIT), Vite (MIT), TypeScript (Apache-2.0), Tailwind CSS (MIT), TanStack Query (MIT), Lucide (ISC).
- **Zero Copyleft Contamination:** Audit confirms zero GPL, AGPL, or viral copyleft licenses across all 21 domain packages.

• **Full Commercial Usability:** All libraries permit commercial distribution, private modification, and SaaS hosting without reciprocity.

5. TECHNICAL IP PROVENANCE & VERIFICATION BASELINE

- **Clean Git Provenance:** Linear commit history originating from initial architectural assembly through baseline commit d5908d0a with unbroken commit provenance.
- **Zero Secrets in Repository:** Dynamic Pydantic Settings ingestion. Comprehensive audit confirms zero active credentials, API keys, or production passwords in Git history.
- **Automated Verification Suite:** Documented historical baseline of 4,260 automated tests across 103 test suites with 99.4% line coverage across core domain logic.

6. COMMERCIAL STATUS & TRANSACTION TERMS FRAMEWORK

- **Proposed Transferable Scope:** Deliverables are proposed for transfer subject to executed definitive transaction agreements. Ownership does not transfer prior to transaction execution and closing.
- **Pre-Revenue Asset:** Revenue, paying customers, and subscribers are NOT ESTABLISHED IN REPOSITORY. Stripe operates strictly in Test Mode with test webhooks.
- **Paper Safety Invariant:** Platform operates with exactly \$0.00 live financial capital at risk under all demonstrated execution paths.